

南開大學

信息隐藏技术课程实验报告

实验五：回声信息隐藏



学 院 密码与网络空间安全学院
专 业 信息安全、法学双学位班
学 号 2313815
姓 名 段俊宇
班 级 信息安全、法学双学位班

一、实验要求

1、 回声信息隐藏:不局限教材中实现回声信息隐藏的方法,建议使用 AI 方法。

2、 图像的位平面:

- 实现对 1-8 任意位平面的提取并显示。
- 实现对 $1 \sim n$ 低位平面的图像显示和 $8 \sim (n+1)$ 高位平面的图像显示。
- 实现去掉 $1 \sim n$ 位平面后的图像的显示。

二、实验原理

1. 回声隐写算法

方法:在数字声音信号中引入回声,可根据引入回声的位置不同来隐藏秘密信息。

原理:掩蔽效应。强信号的存在会使其附近的弱信号难以被感知。人耳对一个较强声音后的极短时间内的较弱声音的不可察觉性。当回声与原声的间隔充分接近时,人耳难以区别回声和原声。

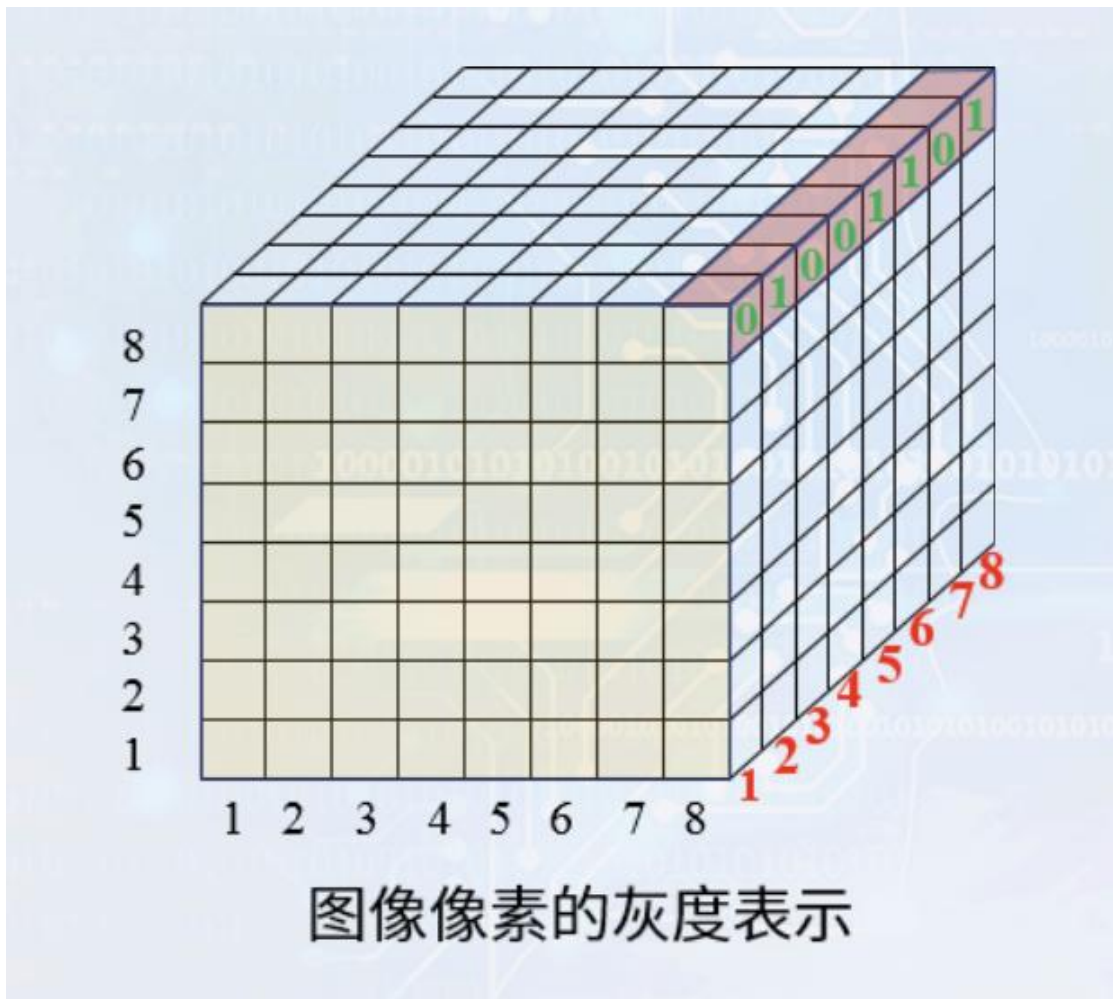
嵌入过程:原始信号为 $f(t)$,回声信号为 $f(t - \Delta t)$,伪装信号为 $c(t) = f(t) + \alpha f(t - \Delta t)$,将信号分段,在每一段中隐藏一比特。

提取过程:对每段信号求自相关函数,自相关函数出现峰值的位置就代表秘密信息比特。

回声信号:可简单模拟为原始信号经过时延和幅度衰减后产生的信号。

2. 图像的位平面

各像素位置相同的位形成一个位平面,不同位平面的重要程度不同,高位平面主要为图像信息,低位平面主要为噪声。



三、实验环境

Matlab_R2025b, Image Processing Toolbox

四、实验过程

1. 回声信息隐藏

如下所示，我将使用注释的方式对代码进行解释。

```
1. clc; clear; close all;
2.
3. % 参数设置
4. SECRET_MSG = 'Hello TOM';
5. AUDIO_FILE = '../Video/test.mp3';
6. OUTPUT_FILE = '../Video/stego_audio.mp3';
7.
8. % 回声参数
9. d0 = 100;
10. d1 = 200;
11. alpha = 0.5;
```

```
12. segment_len = 8192;
13.
14. fprintf('=== 回声信息隐藏系统 ===\n');
15. fprintf('待隐藏消息: %s\n', SECRET_MSG);
16.
17. % 读取音频
18. [audio, fs] = audioread(AUDIO_FILE);
19. audio = double(audio);
20.
21. % 若为立体声, 取左声道
22. if size(audio, 2) > 1
23.     audio = audio(:, 1);
24. end
25.
26. % 消息转换为比特流
27. bits = text_to_bits(SECRET_MSG);
28. fprintf('消息比特数: %d\n', length(bits));
29.
30. % 验证音频是否足够长
31. required_len = length(bits) * segment_len;
32. if length(audio) < required_len
33.     error('音频太短! 需要 %d 样本, 实际 %d 样本', required_len, length(audio));
34. end
35.
36. % AI 自适应延迟选择, 使用局部能量分析自适应选择回声参数, 提高不可感知性
37. fprintf('\n 正在进行 AI 自适应参数优化...\n');
38. [d0_adapt, d1_adapt, alpha_adapt] = adaptive_echo_params(audio, bits, segment_len,
d0, d1, alpha);
39.
40. % 嵌入回声
41. fprintf('正在嵌入回声...\n');
42. stego_audio = embed_echo(audio, bits, segment_len, d0_adapt, d1_adapt,
alpha_adapt);
43.
44. % 保存隐写音频, 归一化防止截幅
45. max_val = max(abs(stego_audio));
46. if max_val > 1
47.     stego_audio = stego_audio / max_val * 0.99;
48. end
49. audiowrite(OUTPUT_FILE, stego_audio, fs);
50. fprintf('隐写音频已保存: %s\n', OUTPUT_FILE);
51.
52. % 提取验证
53. fprintf('\n 正在提取隐藏消息...\n');
```

```

54. [stego_back, fs2] = audioread(OUTPUT_FILE);
55. stego_back = double(stego_back(:,1));
56.
57. extracted_bits = extract_echo(stego_back, length(bits), segment_len, d0_adapt,
d1_adapt);
58. extracted_msg = bits_to_text(extracted_bits);
59.
60. fprintf('提取的消息: %s\n', extracted_msg);
61. if strcmp(extracted_msg, SECRET_MSG)
62.     fprintf('✓ 消息提取成功, 与原始消息完全一致! \n');
63. else
64.     fprintf('X 消息提取有误, 请调整参数. \n');
65. end
66.
67. % 质量评估
68. N = min(length(audio), length(stego_audio));
69. snr_val = compute_snr(audio(1:N), stego_audio(1:N));
70. fprintf('\n 音频质量评估:\n');
71. fprintf('  SNR: %.2f dB\n', snr_val);
72.
73. % 可视化
74. plot_results(audio, stego_audio, bits, extracted_bits, fs, SECRET_MSG,
extracted_msg);
75.
76. fprintf('\n=== 完成 ===\n');
77.
78.
79. % 辅助函数
80.
81. function bits = text_to_bits(text)
82.     % 文本转比特流 (UTF-8, 每字符 8 位)
83.     bytes = uint8(text);
84.     bits = [];
85.     for b = bytes
86.         for k = 7:-1:0
87.             bits(end+1) = bitand(bitshift(b, -k), 1);
88.         end
89.     end
90. end
91.
92. function text = bits_to_text(bits)
93.     % 比特流转文本
94.     text = '';
95.     n = floor(length(bits)/8)*8;

```

```

96.     for i = 1:8:n
97.         byte = 0;
98.         for k = 0:7
99.             byte = byte + bits(i+k) * 2^(7-k);
100.        end
101.        text(end+1) = char(byte);
102.    end
103. end
104.
105. % 根据各段局部能量调整回声参数，低能量段使用较小衰减，高能量段使用较大衰减
106. function [d0a, d1a, aa] = adaptive_echo_params(audio, bits, seg_len, d0, d1, alpha)
107.
108.     num_bits = length(bits);
109.     energies = zeros(1, num_bits);
110.     for i = 1:num_bits
111.         seg = audio((i-1)*seg_len+1 : i*seg_len);
112.         energies(i) = mean(seg.^2);
113.     end
114.
115.     % 归一化能量
116.     e_norm = energies / (max(energies) + 1e-10);
117.
118.     % 自适应衰减系数：能量高时 alpha 小，能量低时 alpha 稍大
119.     alpha_vec = alpha * (0.6 + 0.4 * e_norm);
120.
121.     % 本实现返回全局均值
122.     d0a = d0;
123.     d1a = d1;
124.     aa = mean(alpha_vec);
125.     fprintf(' 自适应 alpha: %.4f (原始: %.4f) \n', aa, alpha);
126. end
127.
128. % 回声嵌入：对每段音频叠加对应延迟的回声
129. function stego = embed_echo(audio, bits, seg_len, d0, d1, alpha)
130.
131.     stego = audio;
132.     num_bits = length(bits);
133.
134.     for i = 1:num_bits
135.         s_idx = (i-1)*seg_len + 1;
136.         e_idx = min(i*seg_len, length(audio));
137.         seg = audio(s_idx:e_idx);
138.
139.         % 选择延迟

```

```

140.         if bits(i) == 0
141.             delay = d0;
142.         else
143.             delay = d1;
144.         end
145.
146.         % 生成回声段
147.         echo_seg = zeros(size(seg));
148.         for j = delay+1:length(seg)
149.             echo_seg(j) = alpha * seg(j - delay);
150.         end
151.
152.         stego(s_idx:e_idx) = seg + echo_seg;
153.     end
154. end
155.
156. % 回声提取：使用倒谱检测回声延迟峰值
157. function bits = extract_echo(stego, num_bits, seg_len, d0, d1)
158.
159.     bits = zeros(1, num_bits);
160.
161.     for i = 1:num_bits
162.         s_idx = (i-1)*seg_len + 1;
163.         e_idx = min(i*seg_len, length(stego));
164.         seg = stego(s_idx:e_idx);
165.
166.         % 计算实倒谱
167.         N = length(seg);
168.         X = fft(seg, N);
169.         log_X = log(abs(X).^2 + 1e-10);
170.         ceps = real(ifft(log_X));
171.
172.         % 比较 d0 和 d1 处的倒谱幅度
173.         c0 = abs(ceps(d0+1));
174.         c1 = abs(ceps(d1+1));
175.
176.         bits(i) = c1 > c0;
177.     end
178. end
179.
180. function snr = compute_snr(original, stego)
181.     noise = stego - original;
182.     snr = 10 * log10(sum(original.^2) / (sum(noise.^2) + 1e-10));
183. end

```

```

184.
185. function plot_results(audio, stego, bits, ext_bits, fs, orig_msg, ext_msg)
186.     t = (0:length(audio)-1) / fs;
187.     ts = (0:length(stego)-1) / fs;
188.
189.     figure('Name','回声信息隐藏结果','Position',[100 100 1200 800]);
190.
191.     % 原始波形
192.     subplot(3,2,1);
193.     plot(t, audio, 'b', 'LineWidth', 0.5);
194.     title('原始音频波形'); xlabel('时间(s)'); ylabel('幅度');
195.     grid on;
196.
197.     % 隐写波形
198.     subplot(3,2,2);
199.     plot(ts, stego, 'r', 'LineWidth', 0.5);
200.     title('隐写音频波形'); xlabel('时间(s)'); ylabel('幅度');
201.     grid on;
202.
203.     % 差异波形
204.     subplot(3,2,3);
205.     N = min(length(audio), length(stego));
206.     plot((0:N-1)/fs, stego(1:N)-audio(1:N), 'g', 'LineWidth', 0.5);
207.     title('差异信号 (回声噪声) '); xlabel('时间(s)'); ylabel('幅度');
208.     grid on;
209.
210.     % 频谱对比
211.     subplot(3,2,4);
212.     NFFT = 2048;
213.     f = (0:NFFT/2) * fs / NFFT;
214.     A_orig = abs(fft(audio(1:min(NFFT,end)), NFFT));
215.     A_stego = abs(fft(stego(1:min(NFFT,end)), NFFT));
216.     plot(f, 20*log10(A_orig(1:NFFT/2+1)+1e-10), 'b'); hold on;
217.     plot(f, 20*log10(A_stego(1:NFFT/2+1)+1e-10), 'r--');
218.     legend('原始','隐写'); title('频谱对比');
219.     xlabel('频率(Hz)'); ylabel('幅度(dB)'); grid on;
220. end

```

这部分代码对 test.mp3 进行处理，隐藏 Hello TOM 这条信息。首先读取音频并将消息转换为比特流，接着使用 AI 自适应方法计算每段的局部能量。在嵌入时，将音频分为多段，每段对应一个比特；在提取时，对每段音频计算实倒谱，峰值大的对应嵌入的比特。最后通过计算 SNR 评估音频失真程度并展示原始和隐藏后的音频波形图。

如下是代码运行的结果，成功嵌入并提取了隐藏消息。

```
=== 回声信息隐藏系统 ===
待隐藏消息: Hello TOM
消息比特数: 72

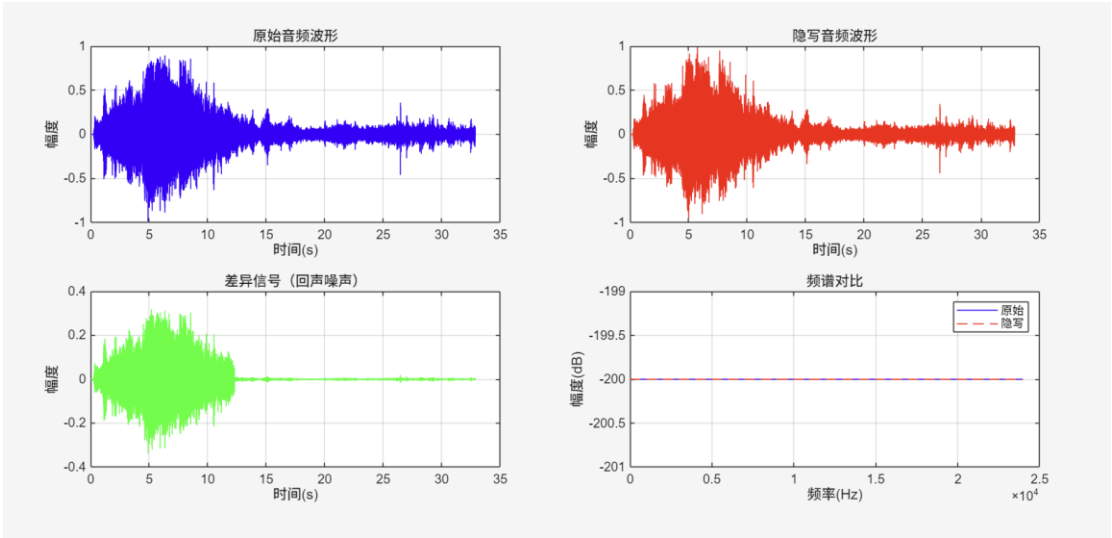
正在进行AI自适应参数优化...
    自适应alpha: 0.3632 (原始: 0.5000)
正在嵌入回声...
隐写音频已保存: ../Video/stego_audio.mp3

正在提取隐藏消息...
提取的消息: Hello TOM
✓ 消息提取成功，与原始消息完全一致！

音频质量评估:
    SNR: 9.48 dB

=== 完成 ===
>>
```

下面是音频波形图对比和回声噪声及频谱对比，从中可以发现隐写后的音频波形与原始音频波形几乎一模一样，实现了良好的隐藏效果。



2. 图像位平面提取

如下所示，我将使用注释的方式对代码进行解释。

```
1. % 图像位平面处理
2. % 实现位平面提取、显示和操作
3. close all;
4. clc;
5.
6. % 加载图像
7. disp('加载图像...');
8. imageFile = '../Picture/1.png';
9. img = imread(imageFile);
10.
11. % 转换为灰度图像
12. if size(img, 3) > 1
13.     img = rgb2gray(img);
14. end
15.
16. % 转换为双精度类型以便处理
17. img = double(img);
18. [m, n] = size(img);
19. disp(['图像大小: ', num2str(m), 'x', num2str(n)]);
20.
21. % 显示原始图像
22. figure;
23. subplot(2, 2, 1);
24. imshow(uint8(img));
25. title('原始图像');
26.
27. % 提取并显示任意位平面
28. function displayBitPlane(img, bit)
29.     bitPlane = bitget(uint8(img), bit);
30.     % 转换为 0-255 范围以便显示
31.     bitPlane = bitPlane * 255;
32.
33.     figure;
34.     imshow(uint8(bitPlane));
35.     title(['第', num2str(bit), '位平面']);
36. end
37.
38. % 显示 1~n 低位平面的图像
39. function displayLowerBitPlanes(img, n)
40.     lowerPlanes = 0;
41.     for i = 1:n
42.         lowerPlanes = lowerPlanes + bitget(uint8(img), i) * 2^(i-1);
43.     end
```

```
44.
45.     figure;
46.     imshow(uint8(lowerPlanes));
47.     title(['1~', num2str(n), '低位平面']);
48. end
49.
50. % 显示(n+1)~8 高位平面的图像
51. function displayUpperBitPlanes(img, n)
52.     upperPlanes = 0;
53.     for i = n+1:8
54.         upperPlanes = upperPlanes + bitget(uint8(img), i) * 2^(i-1); % 权重不变
55.     end
56.
57.     figure;
58.     imshow(uint8(upperPlanes));
59.     title(['第', num2str(n+1), '~8 位高位平面']);
60. end
61.
62. % 显示去掉 1~n 位平面后的图像
63. function displayImageWithoutLowerPlanes(img, n)
64.     result = 0;
65.     for i = n+1:8
66.         result = result + bitget(uint8(img), i) * 2^(i-1);
67.     end
68.
69.     figure;
70.     imshow(uint8(result));
71.     title(['去掉 1~', num2str(n), '位平面后的图像']);
72. end
73.
74. % 显示所有 8 个位平面
75. figure('Name', '所有位平面', 'Position', [100, 100, 1000, 800]);
76. for i = 1:8
77.     bitPlane = bitget(uint8(img), i) * 255;
78.     subplot(2, 4, i);
79.     imshow(uint8(bitPlane));
80.     title(['第', num2str(i), '位平面']);
81. end
82.
83. % 测试低位平面显示
84. disp('测试低位平面显示...');
85. displayLowerBitPlanes(img, 3);
86. displayLowerBitPlanes(img, 5);
87.
```

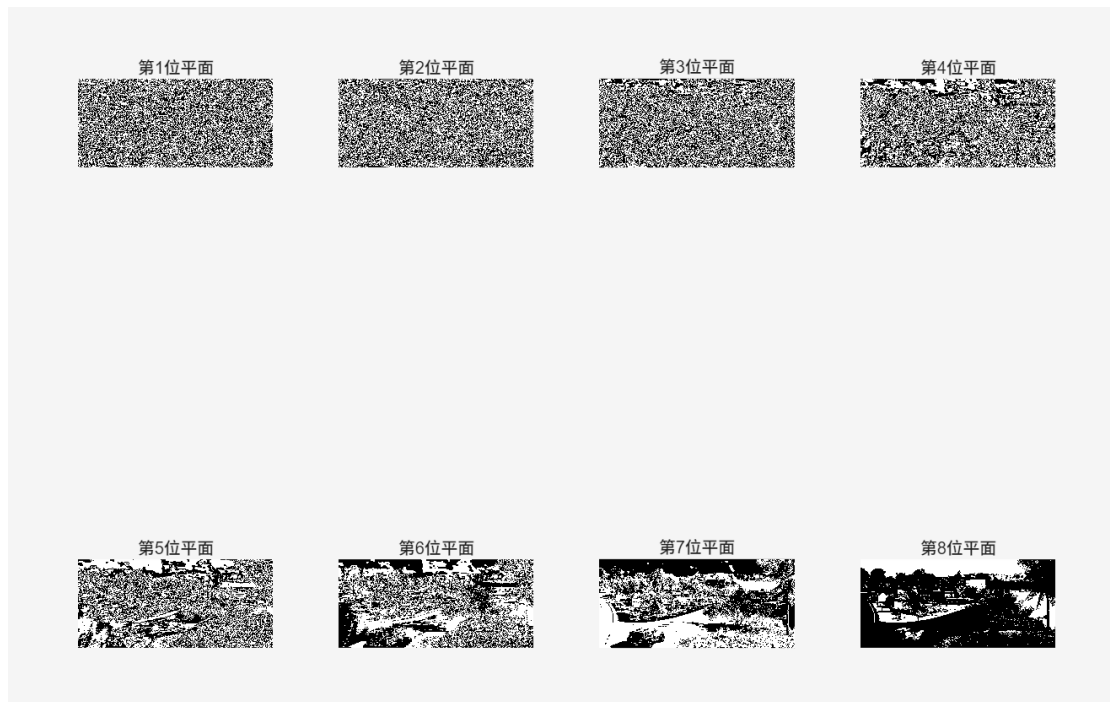
```
88. % 测试高位平面显示
89. disp('测试高位平面显示...');
90. displayUpperBitPlanes(img, 3);
91. displayUpperBitPlanes(img, 5);
92.
93. % 测试去掉低位平面后的图像
94. disp('测试去掉低位平面后的图像...');
95. displayImageWithoutLowerPlanes(img, 3);
96. displayImageWithoutLowerPlanes(img, 5);
97.
98. disp('测试完成! ');
```

这部分代码实现了对图片位平面的提取，通过多个函数分别展现所有位平面图像、低位平面图像、高位平面图像。

下面是原始图像的灰度图像



8 个位平面的图像如下所示，从中可以看出图像的重要信息都在高位平面，而低位平面主要是噪声。



1-3 位平面的叠加图像如下所示，看不出任何信息：



1-5 位平面的叠加图像如下所示，由于高位平面的加入，能够看出一些图形轮廓，但整体还是很暗。

1~5低位平面



6-8 位平面的叠加图像如下所示，可以看出图片的基本信息，但是会出现一些阴影，因为缺少了低位平面的噪声，导致图像光影不够真实。

第6~8位高位平面



去掉 1-3 位平面的图像如下所示，可以看出与原图基本相同。

去掉1~3位平面后的图像



五、实验结论及心得体会

本次实验中我完成了回声隐藏和图像位平面的分离。在没有使用 AI 自适应方法时，提取的信息总是出错，在使用后可以大大提升消息提取的准确性，并且可以在一定程度上扩展消息的长度，其长度也受音频长度的限制。图像位平面的分离让我真实了解到不同位平面包含的信息，实验结果也验证了高位平面包含重要信息，低位平面包含噪声的结论。